Electricity Cost Minimization for Multi-Workflow Allocation in Geo-Distributed Data Centers

Shuang Wang, He Zhang, Tianxing Wu, Yueyou Zhang, Wei Emma Zhang, and Quan Z. Sheng

***Abstract*—Worldwide, Geo-distributed Data Centers (GDCs) provide computing and storage services for massive workflow applications, resulting in high electricity costs that vary depending on geographical locations and time. How to reduce electricity costs while satisfying the deadline constraints of workflow applications is important in GDCs, which is determined by the execution time of servers, power, and electricity price. Determining the completion time of workflows with different server frequencies can be challenging, especially in scenarios with heterogeneous computing resources in GDCs. Moreover, the electricity price is also different in geographical locations and may change dynamically. To address these challenges, we develop a geo-distributed system architecture and propose an Electricity Cost aware Multiple Workflows Scheduling algorithm (ECMWS) for servers of GDCs with fixed frequency and power. ECMWS comprises four stages, namely workflow sequencing, deadline partitioning, task sequencing, and resource allocation where two graph embedding models and a policy network are constructed to solve the Markov Decision Process (MDP). After statistically calibrating parameters and algorithm components over a comprehensive set of workflow instances, the proposed algorithms are compared with the state-of-the-art methods over two types of workflow instances. The experimental results demonstrate that our proposed algorithm significantly outperforms other algorithms, achieving an improvement of over 15% while maintaining an acceptable computational time.**

***Index Terms*—Geo-distributed data centers (GDCs), multiple workflows allocation, electricity cost, graph embedding.**

I. Introduction

With the advancement of global digital transformation, GDCs have gained widespread adoption. These data centers are located in different geographical locations, access and transfer data to each other through the network, and provide efficient and reliable services for users around the world. GDCs have advantages in high fault tolerance and availability, as well as the ability to tailor services based on user requirements [1], [2], [3]. Many applications deployed in cross-domain data centers are represented as directed acyclic graphs (DAGs), which are submitted to systems named workflows. For instance, in scientific computing, Montage [4] is an astronomical image mosaic application that involves a workflow with multiple image processing and data analysis tasks. Empirical data indicate that the global energy expenditure attributed to data centers ranges between 2% and 5%, exhibiting a secular upward trajectory [5]. However, a significant amount of electricity is consumed in cross-domain data centers when executing these applications [6], [7]. Although achievements have been made to improve efficiency [8], [9], how to reduce electricity cost in GDCs is urgent and necessary. In this paper, we consider a multi-workflow scheduling problem for minimizing electricity costs in a cross-domain data center environment.

Electricity pricing varies across geographically dispersed server locations. Concurrently, the processing of tasks on such distributed servers necessitates the consideration of transmission latency, thereby augmenting the complexity of the optimization problem. For workflow scheduling, the orchestration of computational tasks within distributed data centers is complex, particularly when dealing with the constraints of deadlines and the imperative to optimize energy consumption. The challenges associated with resource allocation in such scenarios are multifaceted with the following issues:

- The sharing of resources across data centers for multiple workflows makes it difficult to predict completion time and design algorithms that meet deadline constraints. Resource limitations and heterogeneity among data centers lead to imbalanced task execution on varied servers, complicating the scheduling process.
- The optimization of electricity costs is hindered by the trade-off between the economic use of electricity and the availability of computational resources. Scheduling tasks to cheaper electricity centers may increase execution time, while scheduling to resource-rich centers may reduce execution time but increase electricity costs.
- The competition for resources among workflows intensifies the conflict between resource abundance and electricity cost-effectiveness. Aggressive cost reduction strategies may lead to suboptimal resource allocation, potentially

Received 24 June 2024; revised 17 February 2025; accepted 2 April 2025. Date of publication 22 April 2025; date of current version 12 June 2025. This work was supported in part by National Key Research and Development Program under Grant 2022YFF0902800, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20220803, in part by the NSFC under Grant 62302095, Grant 62376058, and in part by Southeast University Interdisciplinary Research Program for Young Scholars, and the Big Data Computing Center of Southeast University. *(Corresponding author: Tianxing Wu.)*

Shuang Wang, He Zhang, Tianxing Wu, and Yueyou Zhang are with the School of Computer Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: shuangwang@seu.edu.cn; zhanghezhedu@163.com; tianxiangwu@seu.edu.cn; is_zhangyy@163.com).

Wei Emma Zhang is with the School of Computer and Mathematical Sciences, Australian Institute for Machine Learning, The University of Adelaide, Adelaide, SA 5005, Australia (e-mail: wei.e.zhang@adelaide.edu.au).

Quan Z. Sheng is with the School of Computing, Macquarie University, Sydney, NSW 2109, Australia (e-mail: michael.sheng@mq.edu.au).

The source codes are available at https://gitee.com/public-artifacts/ecmws-experiments.

Digital Object Identifier 10.1109/TSC.2025.3562325

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

increasing total electricity cost due to the allocation of tasks to more expensive or slower data centers.

Tackling the aforementioned challenges, the main contributions made in this work are summarized as follows:

- We introduce a novel architecture for scheduling multiple workflows across data centers. It incorporates the heterogeneity of computational resources and the dynamic nature of electricity pricing, leading to the development of foundational mathematical models.
- We propose an Electricity Cost aware Multiple Workflows Scheduling algorithm (ECMWS) to minimize electricity costs under fixed server frequency and power constraints which include four-stage process: workflow sequencing, sub-deadline partitioning, task sequencing, and resource allocation using graph embedding.
- To allocate resources efficiently, we use the Actor-critic framework for workflows by embedding graphs according to which suitable DCs and servers are determined by Deadline Assured Resource Allocation algorithms. The experiments validate that the proposed algorithm outperforms the other algorithms over 15% on the electricity cost in GDCs.

The remainder of this paper is organized as follows. The related research is reviewed and compared in Section II. The mathematical model is defined for the workflow scheduling problem in GDCs in Section III. In Section IV, we present the ECMWS algorithm to minimize the electricity cost. The experimental results with real-world datasets are analyzed and reported in Section V. Finally, the conclusion is summarized in Section VI with several highlighting remarks.

II. Related Work

Scheduling workflows in cross-domain DCs is difficult, as it has been proven to be NP-hard in the previous study [8]. Energy-saving task scheduling in data center scenarios has been widely studied. Wakar et al. [10] pioneered the study of workflow scheduling under budget constraints, aiming to minimize energy consumption. Furthermore, Li et al. [7] under the assumption of fixed bandwidth constraints between DCs, balanced energy cost with electricity prices for scheduling workflows. However, most of these studies [7], [8], [10] ignore the fluctuating electricity prices associated with both location and time for multiple workflows across various geographical locations. In [11], the Dynamic Voltage and Frequency Scaling technology was adopted to save electricity cost by training a policy network for resource allocation while in this paper, we focus on the dynamic price of electricity for geo-distributed data centers with server frequency and power to improve resource allocation.

Compared to the single-workflow scheduling, several studies [12], [13], [14], [15] focused on multi-workflow scheduling that addressed more complex optimization goals, such as total completion time, cost, energy use, and QoS (Quality of Service) for multiple workflows in cloud computing. For instance, Rizvi et al. [12] proposed a fair budget-constrained multi-workflow scheduling algorithm which balanced time and cost by adjusting cost-time efficiency factors. Multi-workflow scheduling is also amenable to application across a variety of computational scenarios. For task scheduling in IoT (Internet of Things), Attiya et al. [13] enhanced the local search by the Salp Swarm Algorithm, which effectively reduced the convergence time of the global optimal solution. In fog-cloud environments, Samia et al. [14] proposed a two-stage scheduling algorithm that minimized workflow energy consumption and makespan. In multi-tenant WaaS (Workflow as a Service) environments, Rodriguez et al. [16] proposed a heuristic dynamic scheduling algorithm catering to both short-term and long-term resource demands of different workflows. These studies provide valuable insights for workflow scheduling in cloud environments. However, the spatio-temporal diversity of electricity prices and the geo-distributed characteristics are ignored.

Deep Reinforcement Learning (DRL) has emerged as a pivotal approach in workflow scheduling [17], [18], [19]. Unlike conventional techniques, DRL is capable of making real-time decisions, which significantly reduces the computational cost. Zhang et al. [17] proposed a real-time workflow scheduling method that combines Genetic Algorithm (GA) with DRL to reduce execution cost and response time. Furthermore, there is a growing interest in DRL-based scheduling algorithms to minimize energy [18], [19]. Yan et al. [18] proposed a DRL based job scheduling algorithm to reduce energy consumption for job execution while keeping high QoS. Additionally, the Actor-Critic framework, a key component in many DRL algorithms, has been widely adopted for dynamic workflow scheduling [19]. Although these methods have shown some promise, they have not fully considered the geographical distributed characteristics on data center locations.

Energy-efficient workflow scheduling problems are related to electricity cost. However, due to the dynamic prices in geo-distributed DCs, scheduling multiple workflows to minimize total electricity cost is more complex than energy minimization problems because of the dynamic electricity characteristic in GDCs. In this paper, we aim to minimize the electricity cost for effectively scheduling multiple workflows in a distributed DC environment for the diverse electricity prices across different locations and time periods.

III. System Framework and Problem Descriptions

For the considered problem, N workflow applications with deadline constraints randomly arrive at the system, i.e., $G = \{G_1, \ldots, G_N\}$. Each workflow $G_w \in G$ is represented as a DAG, denoted as $G_w = (V_w, E_w)$ where $V_w = \{v_1^w, \ldots, v_{n_w}^w\}$ is the set of tasks, n_w is the total number of tasks in workflow G_w, and $v_i^w \in V_w$ is the ith task of workflow G_w. Each task v_i^w has a workload $W_{w,i}$, representing the number of instructions to be executed. Each task v_i^w has direct predecessor and successor task sets PR_i^w and SU_i^w. Note that the PR_i^w set is empty if the task is the first in the workflow. Each task v_i^w needs to receive data produced by its predecessor tasks $v_{i'}^w \in PR_i^w$ before execution, with the data size $S_{i',i}^w$. Table I presents important notations in this paper.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
IMPORTANT NOTATION DEFINITION

Symbol	Definition
N	Total number of workflow applications
M	Total number of data centers
G_w	The wth workflow application
T_w^{submit}	Submission time of application G_w
v_i^w	The ith task in application G_w
n_w	Number of tasks in the workflow G_w
$\mathcal{D}_k$	The kth data center
d_w	Deadline of application G_w
$\mathcal{H}_j^k$	The jth cluster in data center $\mathcal{D}_k$
$\mathcal{V}_{j,l}^k$	The lth server in cluster $\mathcal{H}_j^k$
μ_k	Number of clusters in data center $\mathcal{D}_k$
ω_j^k	Number of servers in cluster $\mathcal{H}_j^k$
$\overline{\omega}$	Number of servers in all data centers
$\xi_{k,j,l}$	Processing speed (MIPS) of servers $\mathcal{V}_{j,l}^k$
$P_{k,j,l}$	Power of servers $\mathcal{V}_{j,l}^k$
$W_{w,i}$	Workload (MI) of task v_i^w
PR_i^w	Set of predecessor tasks of task v_i^w
SU_i^w	Set of successor tasks of task v_i^w
$S_{i',i}^w$	Data volume to be transferred from $v_{i'}^w$ to v_i^w
$T_{w,i}^{work}$	Processing time of task v_i^w
$T_{w,i',i}^{trans}$	Data transmission time from $v_{i'}^w$ to v_i^w
$p_k(t)$	Electricity price of data center $\mathcal{D}_k$ at time t
$C_{w,i}$	Electricity cost of task v_i^w
Z	Electricity cost of all workflows
$T_{w,i}^B$	Start time of task v_i^w
$T_{w,i}^F$	Finish time of task v_i^w
$x_{w,i;k,j,l}$	Binary variable for task v_i^w is allocated to $\mathcal{V}_{j,l}^k$

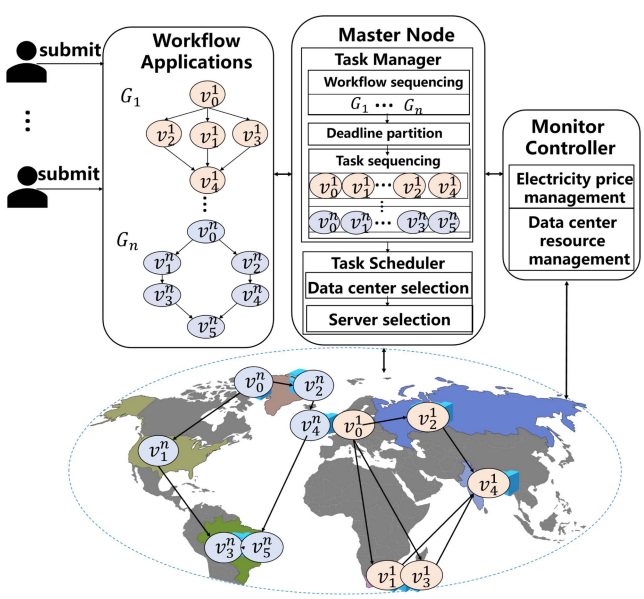


Fig. 1. System architecture.

A. System Architecture

To minimize the electricity cost, the system architecture of multiple workflows scheduling in geo-distributed DCs is shown in Fig. 1. The primary node receives multiple workflows submitted by users at each time slot, and at the end of each slot, the primary node schedules all workflows in the previous slot. The primary node has two modules: *Task Manager* and *Task Scheduler*. The Task Manager sorts a batch of workflows to be scheduled in each slot (Workflow Sequencing), divides the sub-deadlines (Deadline Partition) for each task in the workflow sequentially and determines the scheduling sequence (Task Sequencing). The Task Scheduler sequentially selects a DC and a server for each task in the scheduling sequence. The Monitor Controller is responsible for managing resources in the geo-distributed DCs (DC Resource Management) and the electricity prices in corresponding locations (Electricity Prices Management). After the tasks are executed in the geo-distributed DCs, the performance results are returned to the users through the primary node.

We assume that the service provider operates M DCs: $D = \{D_1, \ldots, D_M\}$, where a DC $D_k \in D$ has an electricity price $p_k(t)$ that periodically changes with time t. The kth DC $\mathcal{D}_k$ contains μ_k clusters, where the jth cluster is represented as H_j^k. Cluster $\mathcal{H}_j^k$ has ω_j^k servers. Let ω^k denote the total number of servers in each DC: $\omega^k = \sum_{j=1}^{\mu_k} \omega_j^k$. $\overline{\omega}$ is denoted as the total number of servers in the system: $\overline{\omega} = \sum_{k=1}^{M} \omega^k$. The processing frequency and power of a server $\mathcal{V}_{j,l}^k$ are represented as $\xi_{k,j,l}$ and $P_{k,j,l}$, respectively. The runtime of the system is computed by a series of continuous slots. During each time slot, the primary node accepts the workflows and schedules all these workflows at the end of each time slot.

B. Problem Formulation

Each application $G_w \in G$ has a submission time T_w^{submit} with a user-specified deadline d_w. Let $T_{w,i}^B$ be the start time of v_i^w, and $T_{w,i}^F$ be the finish time of v_i^w. Each task v_i^w can only start executing after all predecessor tasks $v_{i'}^w \in PR_i^w$ have finished, which should satisfy the constraint:

$$T_{w,i}^B \geq T_{w,i'}^F, i' \in PR_i^w \tag{1}$$

In this paper, we consider multiple workflow applications with deadline constraints which arrive randomly and the heterogeneity of data center resources with dynamic electricity prices. To describe the problem clearly, we have the following assumptions: (i) The server will not fail during task execution. (ii) The task is processed only on the assigned server. (iii) There is no performance fluctuation when the server executes the task, and its processing frequency and power remain constant. (iv) The bandwidth of different data centers, clusters and servers are randomly refreshed in each time step. (v) The energy is consumed by servers during task execution and data transmission.

Workflows are scheduled on different servers of clusters in GDCs. The binary decision variable $x_{w,i;k,j,l}$ is used to denote whether an arbitrary task v_i^w is scheduled to the server $\mathcal{V}_{j,l}^k$ or not. Since a task is executed by one server, for each task v_i^w, it satisfies: $\sum_{k=1}^{M} \sum_{j=1}^{\mu_k} \sum_{l=1}^{\omega_j^k} x_{w,i;k,j,l} = 1$. For each server $\mathcal{V}_{j,l}^k$, any two tasks assigned to it, v_i^w and $v_{i'}^{w'}$, should have non-overlapping execution time with the following constraint: $(x_{w,i;k,j,l} \times T_{w,i}^B \geq x_{w',i';k,j,l} \times T_{w',i'}^F) \vee (x_{w,i;k,j,l} \times T_{w,i}^F \leq x_{w',i';k,j,l} \times T_{w',i'}^B)$. The finish time of a task, $T_{w,i}^F$, is determined by the start time, $T_{w,i}^B$, and the execution time of task v_i^w on the server $T_{w,i}^D$, which includes the maximum

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

transmission time from the predecessor task to the current task and the execution time, $T^{work}_{w,i}$, to complete the workload $W_{w,i}$:

$$T^F_{w,i} = T^B_{w,i} + T^D_{w,i} \tag{2}$$

where $T^D_{w,i} = T^{work}_{w,i} + \max_{v^w_{i'} \in PR^w_i} T^{trans}_{w,i',i}$ and $T^{work}_{w,i} = \sum_{k=1}^{M} \sum_{j=1}^{\mu_k} \sum_{l=1}^{\omega^k_j} x_{w,i;k,j,l} \times \frac{W_{w,i}}{\xi_{k,j,l}}$.

The workload $W_{w,i}$ is measured in Million of Instructions (MI), and the processing frequency of server $\xi_{k,j,l}$ is in MIPS (MI Per Second). The transmission time for each predecessor task $v^w_{i',i} \in PR^w_i$ to v^w_i is $T^{trans}_{w,i',i}$. When tasks $v^w_{i'}$ and v^w_i are assigned to the same cluster, the transmission time is zero because data transfers locally. Otherwise, the transmission time is calculated by the bandwidth B_{in} for different clusters within the same data center or B_{out} for transmission between different centers. We introduce a ternary auxiliary variable e to represent the different data transmission. The transmission time $T^{trans}_{w,i',i}$ is calculated by:

$$T^{trans}_{w,i',i} = \frac{e_{i',i} S_{w,i',i}}{2(e_{i',i} - 1)B_{out} + (2 - e_{i',i})B_{in}}$$

$$e_{i',i} = \begin{cases} 0, & \text{if servers of } \mathrm{v^w_{i'}} \text{ and } \mathrm{v^w_i} \text{ are in the same cluster;} \\ 1, & \text{if servers are in the same} \\ & \text{DC but different clusters;} \\ 2, & \text{if servers are in different DCs.} \end{cases} \tag{3}$$

The finish time of the latest completed task of workflow G_w should not exceed d_w: $\max_{i \in \{1,\ldots,n_w\}} \{T^F_{w,i}\} \le d_w$.

The objective is to minimize the electricity cost for workflow applications, which is closely related to the electricity price $p_k(t)$ of geo-distributed DCs, the execution time $T^D_{w,i}$ of the task on the server, and the power consumption $P_{k,j,l}$ of the server. The mathematical model is described as follows:

$$\min \tilde{Z} = \sum_{w=1}^{N} \sum_{i=1}^{n_w} C_{w,i} \tag{4}$$

$$C_{w,i} = \sum_{k=1}^{M} \sum_{j=1}^{\mu_k} \sum_{l=1}^{\omega^k_j} \int_{T^B_{w,i}}^{T^F_{w,i}} x_{w,i;k,j,l} \times P_{k,j,l} \times p_k(t) dt \tag{5}$$

$$\min_{i=1,\ldots,n_w} \{T^B_{w,i}\} \ge T^{submit}_w \tag{6}$$

$$\max_{i=1,\ldots,n_w} \{T^F_{w,i}\} \le d_w \tag{7}$$

IV. Electricity Cost-Aware Workflow Scheduling

To minimize the electricity cost from geo-distributed DCs, we propose the Electricity Cost-aware Multiple Workflows Scheduling (ECMWS) algorithm, which considers servers with fixed frequency and power similar as studies [7], [10], [20], [21]. The framework of ECMWS is shown in Algorithm 1 with termination time and scheduling intervals. First, for various requirements of distinct workflows, to avoid repeated calculation with un-satisfied servers and improve algorithm efficiency, the performance parameters of servers such as the average processing frequency and the transmission bandwidth are pre-estimated.

Algorithm 1: ECMWS Framework.

Input: System termination time T^{term}, interval τ
Output: Electricity cost Z
$Z \leftarrow 0$, $t \leftarrow 0$;
Estimate VM performance parameters;
while $t < T^{term}$ **do**
  $t' \leftarrow \min\{t + \tau, T^{term}\}$;
  $G \leftarrow \emptyset$;
  $G \leftarrow G \cup workflows$ submitted between t to t';
  if $G \neq \emptyset$ **then**
    Estimate time parameters of tasks for workflows;
    $(G_1, G_2, \ldots, G_{N'}) \leftarrow CWS(G)$;
    for $w = 1$ *to* N' **do**
      $\{(v^w_1, d_{w,1}), \ldots, (v^w_{n_w}, d_{w,n_w})\} \leftarrow BLDP(G_w)$.
      $(v^w_1, v^w_2, \ldots, v^w_{n_w}) \leftarrow TS(G_w)$;
      for $i = 1$ *to* n_w **do**
        $(D_k, V^k_{j,l}) \leftarrow CCRA(v^w_i)$;
        Compute $C_{w,i}$ by Eq. (5).
        $Z \leftarrow Z + C_{w,i}$;
  $t \leftarrow t'$;

Second, due to the imperative to ensure the completion of all workflows prior to their respective deadlines, it is essential to implement a scheduling mechanism that prioritizes and sequences the workflows. ECMWS collects workflows submitted to the system, estimates the execution and transmission time of tasks in workflows, and we propose the Contention aware Workflow Sequencing (CWS) algorithm to get workflow scheduling sequence in Algorihtm 2. Third, given the interdependent relationship among tasks within workflows, it is essential to effectively allocate resources by implementing a prioritization and sequencing process for tasks as well. Thus, for each workflow in the scheduling sequence, the Bottleneck-Layer-aware sub-Deadline Partitioning (BLDP) and Task Sequencing (TS) procedures are proposed to obtain the scheduling sequence of tasks. For resource allocation, to mitigate the overall electricity cost associated with task execution while avoiding entrapment in local optima, we train an electricity cost-aware policy network. For each task in the task scheduling sequence, ECMWS performs a Confidence Constrained Resource Allocation (CCRA) algorithm to allocate DCs and servers to it by graph embedding and Proximal Policy Optimization (PPO). By repeating the above procedures at fixed scheduling intervals, the electricity cost is minimized.

To illustrate the ECMWS framework, we assume that there are two workflows to be allocated: workflow 1 with 17 tasks and workflow 2 with 6 tasks with 1 h deadline. We follow the common setting in data center platforms, like Cisco Nexus 5600.[1] There are three cross-domain data centers with electricity price (0.16$all day, and 0.16$(0:00-8:00), 0.19$(8:01-23:59)). Each data center has two clusters, and each cluster has two servers. The processing frequencies of servers are 1,000 MIPS and 500 MIPS, respectively. To better simulate the actual network scheduling, we set up dynamic bandwidths for different

[1]https://www.cisco.com/c/en/us/products/collateral/switches/nexus-5624q-switch/datasheet-c78-733100.html

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

states. When the network is in a congested state, the actual bandwidth would be randomly reduced [22] (30%-70% of the corresponding bandwidth in this paper). The bandwidth between data centers is randomly changed between 0 to 100 Gbps, while the bandwidth between clusters in a data center is set to 0 to 80 Gbps. All data transmission between servers within the same cluster have a consistent random bandwidth between 0 to 40 Gbps. The specific scheduling process with the same example is discussed in the following subsections.

A. Parameter Estimation

For geo-distributed DCs, there are exponential growth combinations for multiple workflows with heterogeneous servers. To avoid redundant computations and enhance the efficiency, ECMWS estimates the parameters of servers during the initialization phase including the average processing frequency of servers and the average data transmission bandwidth between servers. The former is defined as the average processing frequency of all servers in cross-domain DCs where $\hat{\xi} = \frac{\sum_{k=1}^{M}\sum_{j=1}^{\mu_k}\sum_{l=1}^{\omega_j^k}\xi_{k,j,l}}{\overline{\omega}}$. The latter is the average of the combined transmission bandwidth of all pairwise virtual groups:

$$\overline{B} = \frac{B_{in} \times \sum_{k=1}^{M}\sum_{j=1}^{\mu_k-1}\sum_{j'=j+1}^{\mu_k}(\omega_j^k \times \omega_{j'}^k)}{\overline{\omega}^2} + \frac{B_{out} \times \sum_{k=1}^{M-1}\sum_{k'=k+1}^{M}(\omega^k \times \omega^{k'})}{\overline{\omega}^2} \tag{8}$$

where $\omega^k = \sum_{j=1}^{\mu_k}\omega_j^k$. Given a task v_i^w, since the task may be assigned to any server, the average processing frequency of server $\overline{\xi}$ is used to estimate the actual execution time of v_i^w:

$$\hat{T}_{w,i}^{work} = \frac{W_{w,i}}{\hat{\xi}} \tag{9}$$

The average data transfer bandwidth $\overline{B}$ between servers $\hat{T}_{w,i}^{trans} = \max_{v_{i'}^w \in PR_i^w}\{\hat{T}_{w,i',i}^{trans}\}$ is used to estimate the data transmission time of the task,

$$\hat{T}_{w,i',i}^{trans} = \frac{S_{i',i}^w}{\overline{B}} \tag{10}$$

where $\hat{T}_{w,i',i}^{trans}$ is the data transmission time of task v_i^w and its immediate predecessor task $v_{i'}^w$.

According to the task execution time and data transmission time, the earliest start time $\hat{T}_{w,i}^{EST}$ and the earliest finish time $\hat{T}_{w,i}^{EFT}$ of each task is computed by a dynamic programming method [23] based on the execution time $\hat{T}_{w,i}^{work}$ and data transmission time $\hat{T}_{w,i}^{trans}$.

$$\hat{T}_{w,i}^{EFT} = \hat{T}_{w,i}^{EST} + \hat{T}_{w,i}^{trans} + \hat{T}_{w,i}^{work} \tag{11}$$

$$\hat{T}_{w,i}^{EST} = \begin{cases} \lceil \frac{T_w^{submit}}{\tau} \rceil \times \tau, & \text{if } PR_i^w = \emptyset \\ \max_{v_{i'}^w \in PR_i^w}\left\{\hat{T}_{w,i'}^{EFT}\right\}, & \text{else} \end{cases} \tag{12}$$

According to (1), (12) indicates that the earliest start time of each task is determined by its direct predecessor tasks with the latest completion time. The parameter estimation consists of two parts: *initialization* and *scheduling* in Algorithm 1. For all N workflows submitted to the system, the time complexity of parameter estimation is $O(\overline{\omega}^2 + \sum_{w=1}^{N} n_w^2)$. Parameter estimation is essential for assessing workflow workloads and reducing the resource waste generated by un-suitable servers. In multiple workflow environments, it guides the order of execution of tasks that can be used in resource allocation to improve overall efficiency.

B. Workflow Sequencing

Optimal scheduling results are achieved if the system has abundant resources while resources are usually limited, which leads to local optimal when optimizing electricity costs for multiple workflow allocations. Interleaved scheduling struggles to meet requirements and constraints within limited computational complexity. Therefore, a reasonable workflow sorting strategy is applied based on their contention where the CWS algorithm is proposed. CWS determines the workflow sequence defined by the workload factor $WL(G_w)$, slack time $ST(G_w)$, and resource contention factor $CT(G_w)$ for G_w. $WL(G_w)$ is the normalized value of the sum of all workflow tasks W_w:

$$WL(G_w) = \frac{W_w}{\max_{w'=1,\ldots,N'} W_{w'}} \tag{13}$$

where $W_w = \sum_{i=1}^{n_w} W_{w,i}$. The slack time factor $ST(G_w)$ is denoted as the normalized value of the workflow slack time:

$$ST(G_w) = \frac{T_w^{slack}}{\max_{w'=1,\ldots,N'} T_{w'}^{slack}} \tag{14}$$

where the slack time T_w^{slack} is the difference between the earliest completion time $\hat{T}_w^{EFT}$ and deadline d_w: $T_w^{slack} = d_w - \hat{T}_w^{EFT}$. The smaller slack time indicates that the application is more urgent. $\hat{T}_w^{EFT}$ is determined by the completion time of last task on its critical path: $\hat{T}_w^{EFT} = \max_{i=1,\ldots,n_w}\hat{T}_{w,i}^{EFT}$. The resource contention factor $CT(G_w)$ is the normalized value of the workflow resource contention $Contention_w$:

$$CT(G_w) = \frac{Contention_w}{\max_{w'=1,\ldots,N'} Contention_w} \tag{15}$$

where $Contention_w$ represents the maximum number of servers that the workflow may occupy simultaneously. To calculate CWS, the procedures are described in Algorithm 2. First, the execution time interval of each task from $\hat{T}_{w,i}^{EST}$ to $\hat{T}_{w,i}^{EFT}$ is added to the queue $Interval$ in ascending order of $\hat{T}_{w,i}^{EST}$. The number of overlapping time intervals $curContention$ of a task with other later tasks is counted by traversing $Interval$ with maximum value $Contention_w$. The $ComputeContention$ function is used to compute the resource contention level $Contention_w$ and the contention factor $CT(G_w)$. To balance these factors, CWS defines the rank of a workflow application G_w as the weighted sum of the factors measured by parameters $\alpha_1, \alpha_2, \alpha_3 \in [0, 1]$:

$$Rank(G_w) = \alpha_1 ST(G_w) + \alpha_2 WL(G_w) + \alpha_3 CT(G_w) \tag{16}$$

To reduce the complexity of parameter correction by shrinking the parameter space, the following constraint should be satisfied

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: CWS.

Input: workflow set G, weight parameters $\alpha_1, \alpha_2, \alpha_3$
Output: workflow scheduling sequence Seq
$R \leftarrow \emptyset$;
for $G_w \in G$ **do**
  Calculate $WL(G_w)$ according to Eq. (13) ;
  Calculate $ST(G_w)$ according to Eq. (14);
  $Contetion_w \leftarrow$ ComputeContention(G_w) ;
  Calculate $CT(G_w)$ according to Eq. (15);
  Calculate $Rank_w$ by Eq. (16);
  $R \leftarrow R \cup (G_w, Rank_w)$;
$Seq \leftarrow$ Sort workflows in G ascendingly based on R;

without losing generality:

$$\alpha_1 + \alpha_2 + \alpha_3 = 1. \tag{17}$$

Finally, the rank $Rank_w$ is calculated, and the scheduling sequence is obtained in the ascending order of rank. Given a total of T batches of workflows to be scheduled, the size of each batch $\{N_1, \ldots, N_T\}$ and the constraint condition $\sum_{t=1}^{T} N_t = N$, the total time complexity of the workflow scheduling algorithm is $O(\sum_{w=1}^{N} n_w \log n_w + \sum_{t=1}^{T} N_t \log N_t)$.

For example, when $\alpha_1 = 0.4, \alpha_2 = 0.2$ and $\alpha_3 = 0.4$, according to (13), (14), (15), these parameters are calculated as follows: $WL(G_1) : 0.72, WL(G_2) : 0.28, ST(G_1) : 0.30, ST(G_2) : 0.16, CT(G_1) : 1, CT(G_2) : 1$. Thus, the workflow scheduling sequence is computed by (16) with $Seq = \{G_1, G_2\}$.

C. Deadline Partition

To improve the efficiency and fairness of workflow allocation, workflow deadlines should be refined into sub-deadlines of each task. By appropriately assigning sub-deadlines to each task, all tasks are completed within the partition deadline meanwhile satisfying the workflow deadline constraint [24]. With limited resources, if the sub-deadlines of tasks are too wide or too tight, it will lead to low resource utilization and long completion time of workflow which may violate the deadline constraints. To comprehend the dependency among tasks in the workflow, tasks are classified into various levels, where tasks within the same level can be executed in parallel, while tasks with different levels exhibit dependency relationship.

We propose a BLDP strategy to partition sub-deadlines for a workflow by calculating the rank of tasks. First, it calculates task ranks by the upstream completion time [25]. Second, BLDP partitions each task v_i^w in G_w into a level $l_{w,i}$ based on its dependencies and the number of tasks $\phi(l_{w,i})$ at the same level in a workflow. $\phi(l_{w,i}) = |\{v_j^w | l_{w,j} = l_{w,i}\}|$ where $l_{w,i} = 1$ if $SU_i^w = \emptyset$, otherwise $l_{w,i} = \max_{v_j^w \in SU_i^w} l_{w,i} + 1$. To compute the bottleneck-layer-aware rank $Rank_{w,i}^{DP}$ for v_i^w, we use the estimated task processing time $\hat{T}_{w,i}^{work}$ by (9), and the estimated data transmission time $T_{w,i,j}^{trans}$ by (10). The rank is calculated by:

$$Rank_{w,i}^{DP} = \begin{cases} \hat{T}_{w,i}^{work}, & \text{if } SU_i^w = \emptyset \\ \max_{v_j^w \in SU_i^w} (Rank_{w,j}^{DP} + \hat{T}_{w,i,j}^{trans}) & \\ + \hat{T}_{w,i}^{work} + \beta^{\frac{\phi(l_{w,i})}{\phi(l_{w,i}-1)}}, & \text{else} \end{cases} \tag{18}$$

Finally, the sub-deadline $d_{w,i}$ for each task v_i^w is calculated by its rank and the workflow deadline d_w.

$$d_{w,i} = d_w \times \frac{Rank_{w,0}^{DP} - Rank_{w,i}^{DP} + \hat{T}_{w,i}^{work}}{Rank_{w,0}^{DP}} \tag{19}$$

$$Rank_{w,0}^{DP} = \max_{\substack{1 \le i \le n_w \\ PR_i^w = \emptyset}} Rank_{w,i}^{DP} \tag{20}$$

Based on above analysis, the sub-deadline for each task of workflow 1 is calculated as follows: $d_{1,1} = 0.12, d_{1,2} = 0.16, d_{1,3} = 0.23, \ldots, d_{1,17} = 1$.

D. Task Sequencing

The task scheduling sequence has a significant impact on the electricity cost incurred during the task scheduling process. Given a workflow G_w and a task $v_i^w \in G_w$, the upward rank $Rank_{w,i}^{up}$ and downward rank $Rank_{w,i}^{down}$ are calculated by the upstream and downstream completion times [25], respectively. We construct the task scheduling sequence by sorting v_i^w in descending order according to $Rank_{w,i}^{up}$ or in ascending order according to $Rank_{w,i}^{down}$. The definitions of the two types of task rank are described as follows:

$$Rank_{w,i}^{up} = \max_{v_j^w \in SU_i^w} \hat{T}_{w,i,j}^{trans} + Rank_{w,j}^{up} + \hat{T}_{w,i}^{work} \tag{21}$$

$$Rank_{w,i}^{down} = \begin{cases} 0, & \text{if } PR_i^w = \emptyset \\ \max_{v_j^w \in PR_i^w} (\hat{T}_{w,j,i}^{trans} & \\ + \hat{T}_{w,j}^{work} + Rank_{w,i}^{down}), & \text{else} \end{cases} \tag{22}$$

We reuse the rank $Rank_{w,i}^{DP}$ by (18) to ensure that tasks with higher ranks are executed earlier than tasks with lower ranks. Tasks are sorted as follows: (1) TS_1: Calculate the upward rank $Rank_{w,i}^{up}$ for each task v_i^w by (21) and sort in decreasing order. (2) TS_2: Calculate the downward rank $Rank_{w,i}^{down}$ for each task v_i^w by (22) and sort in increasing order. (3) TS_3: Calculate the task rank $Rank_{w,i}^{DP}$ for each task v_i^w by (18) and sort in decreasing order.

Since computing $Rank_{w,i}^{up}$ and $Rank_{w,i}^{down}$ requires examining every combination of a task with all direct predecessors or successors, the total time complexity of TS_1 and TS_2 for N workflows is $O(\sum_{w=1}^{N} n_w^2)$ while the time complexity of TS_3 is $O(\sum_{w=1}^{N} n_w \log n_w)$, which only requires sorting the workflow tasks based on the task rank values computed in the previous stage. According to TS_3, the task sequence of workflow 1 is $(v_1^1, v_2^1, v_4^1, v_3^1, v_5^1, v_6^1, v_7^1, v_9^1, v_8^1, v_{10}^1, v_{11}^1, v_{12}^1, v_{13}^1, v_{15}^1, v_{14}^1, v_{17}^1, v_{16}^1)$, and for workflow 2, it is $(v_6^2, v_5^2, v_2^2, v_4^2, v_3^2, v_1^2)$.

E. Resource Allocation

For task resource allocation, the objective is to allocate servers effectively to reduce the electricity cost of task execution while

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: CCRA.

Input: Task v_i^w; Workflow G_w; Confidence threshold $Conf_{thresh}$
Output: DC D_k and server $V_{j,l}^k$ assigned to v_i^w
Construct $model_t$ and $model_r$;
$\vec{S}_{task} \leftarrow model_t(v_i^w, G_w)$;
$\vec{S}_{server} \leftarrow model_r(D_1, \ldots, D_M)$;
Obtain Policy network π_θ by RAPPO in Algorithm 4;
Construct the price state vector $\vec{S}_{price}$ by Eq. (23);
Construct $\vec{S}_{params}$ by Eq. (24);
$\vec{S}_t \leftarrow [\vec{S}_{task} \quad \vec{S}_{server} \quad \vec{S}_{price} \quad \vec{S}_{params}]$;
$Pr_{A|S} \leftarrow \pi_\theta(\vec{S}_t)$;
$A_t \leftarrow \arg\max_A \left\{ Pr_{A|\vec{S}_t} \right\}$;
$Conf_t \leftarrow \max_A \left\{ Pr_{A|\vec{S}_t} \right\}$;
if $Conf_t \geq Conf_{thresh}$ **then**
  $(D_k, V_{j,l}^k) \leftarrow A_t$;
else
  $(D_k, V_{j,l}^k) \leftarrow DARA(v_i^w)$;

satisfying the task sub-deadline constraints. However, solely minimizing the electricity cost incurred by individual tasks can lead to local optimal optimization of the overall electricity cost for all workflows. Therefore, the Proximal Policy Optimization (PPO) algorithm [26] is employed to train an electricity-cost-aware policy network. This policy network takes a global view of the batch workflow scheduling process and dynamically assigns each task to the optimal data center and the specific server based on the real-time system state, approximating the overall scheduling objective optimally. Subsequently, a CCRA algorithm is proposed based on this policy network, which incorporates confidence constraints into the task resource allocation process. In Algorithm 3, the task and resource graph embedding models are constructed to generate the state vectors, respectively. These vectors are used as input to call the policy network to generate the probability distribution of resource allocation actions, and the action with the highest probability is selected. If the probability of the selected action is no less than a given threshold, the DC and server selected by this action are adopted. Otherwise, a fallback policy is executed to select a new DC and server.

For workflows $G = \{G_1, \ldots, G_N\}$, the scheduling process involves decision points of tasks assignment. The system state $S_t \in \mathcal{S}$ at time step t is considered as input, and the policy network is trained to generate resource allocation actions $A_t \in \mathcal{A}$. By constructing Markov Decision Process (MDP) that encompasses $\overline{n} = \sum_{G_w \in G} n_w$ time steps for geo-distributed DCs, the electricity cost is minimized with multiple workflows. Each component of the MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma, \rho_0)$ is described as follows.

$\mathcal{S}$: a continuous space, where each state $S_t \in \mathcal{S}$ is constructed by concatenating the task embedding vector $\vec{S}_{task}$, resource embedding vector $\vec{S}_{server}$, electricity price vector for cross-domain DCs $\vec{S}_{price}$, and parameter vector of the algorithm $\vec{S}_{params}$. $\vec{S}_{price}$ is constructed by concatenating the normalized values of the average electricity prices for each hour in future, Dim_{price} at each DC. These prices are calculated by the current hour h (represented in UTC time) of the cross-domain DC system:

$$\vec{S}_{price} = \frac{\vec{p} - \vec{p}_{min}}{\vec{p}_{max} - \vec{p}_{min}} \tag{23}$$

where $\vec{p} = \begin{bmatrix} \hat{p}_1(h), \ldots, \hat{p}_1(h + Dim_{price} - 1), \ldots, \\ \hat{p}_M(h), \ldots, \hat{p}_M(h + Dim_{price} - 1) \end{bmatrix}$, $\vec{p}_{min} = \left[\overbrace{p_{min} \quad \cdots \quad p_{min}}^{D_{price}}\right]$, $\vec{p}_{max} = \left[\overbrace{p_{max} \quad \cdots \quad p_{max}}^{D_{price}}\right]$, $p_{min} = \min_{\substack{k=1,\ldots,M \\ t=0,\ldots,24\times3600}} p_k(t)$, $p_{max} = \max_{\substack{k=1,\ldots,M \\ t=0,\ldots,24\times3600}} p_k(t)$, and $\hat{p}_k(h) = \frac{\sum_{t=h\times3600}^{(h+1)\times3600} p_k(t)}{3600}$. $\vec{S}_{params}$ is embedded into the State Space by CWS, BLDP, and Task sorting Parameters:

$$\vec{S}_{params} = \left[\overbrace{\alpha_1 \quad \alpha_2 \quad \alpha_3}^{\text{CWS parameters}} \quad \overbrace{\beta}^{\text{BLDP parameter}} \quad \overbrace{0 \quad 0 \quad 1}^{\text{Task sorting}}\right] \tag{24}$$

$\mathcal{A}$: For a discrete action space, each action ($A_t \in \mathcal{A}$) denotes the allocation of a DC and a server to a task. $\mathcal{A}$ is the Cartesian product of two discrete sub-action spaces: $\mathcal{A} = \mathcal{A}_M^{dc} \times \mathcal{A}_{\omega_{max}}^{server}$. $\mathcal{A}_M^{dc}$ represents the action space for selecting a DC where M is the number of DCs. $\mathcal{A}^{server}$ represents the action space for selecting a server, where $\omega_{max} = \max_{k\in1,\ldots,M}(\sum_{j=1}^{\mu_k} \omega_j^k)$ is the maximum number of servers across all DCs. The policy network samples from these two action spaces to generate sub-actions for selecting a DC and selecting a server, respectively.

$\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition function, which provides the probability of transitioning from current state to next state given a particular action. The PPO algorithm is employed to optimize the policy network. $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function with real-valued outputs. Based on the optimization objective and the deadline constraints of the workflow, the following reward function is designed for the task v_i^w to be scheduled at time step t:

$$R_t = -C_{w,i} \times \left(1 + \max\left(0, \frac{T_{w,i}^F - d_{w,i}}{T_{w,i}^F - T_{w,i}^B}\right)\right) \tag{25}$$

The task's electricity cost $C_{w,i}$ and finishing time $T_{w,i}^F$ are computed by (5) and (2), respectively. The objective of $\mathcal{M}$ is to learn the optimal policy network parameters θ. At each time step t, the policy network π_θ is utilized to select the best DC and server for state s, thereby maximizing the expected cumulative reward $V^\pi(s)$: $V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$. γ: The discount factor is used to calculate the discounted cumulative reward G_t, which evaluates the long-term value of taking an action at a particular time step:

$$G_t = \sum_{i=0}^{\overline{n}-t-1} \gamma^i R_{t+i}. \tag{26}$$

ρ_0: initial state distribution. For the scheduling process of different batch workflows, the characteristics of the first task of workflows to be scheduled and the availability of server resources varies, resulting in different initial states of $\mathcal{M}$.

1) Graph Embedding Model for Tasks and Resources: According to the partial order relation constraint of tasks in workflows in (1), the structure of workflow is crucial to the scheduling of workflow tasks. Graph Convolutional Network (GCN) only

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

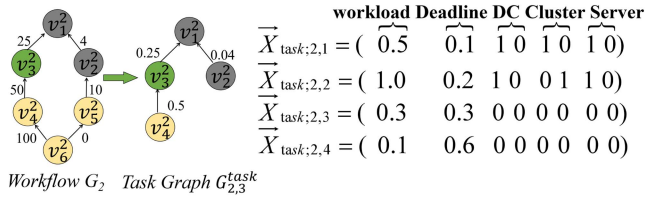


Fig. 2. Task diagram example.

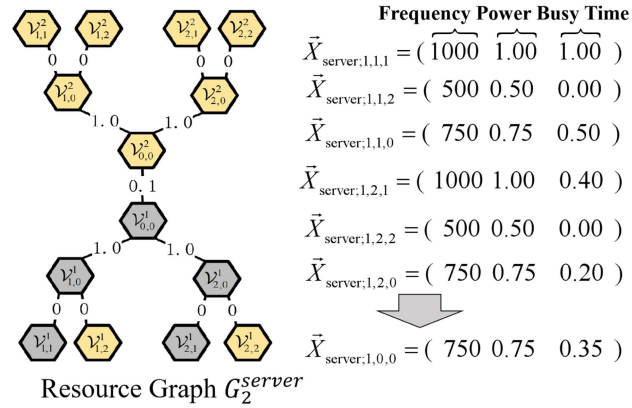


Fig. 3. Resource diagram example.

considers the local neighborhood information of the node, which enables the model to capture the local connection pattern of the node more efficiently. By employing a graph embedding model $model_t$ with two-layer GCN, the high-dimensional structured workflow task information is encoded into a low-dimensional vector, where the output dimensions are D_{tg}, D_{tn} in the Encoder, and D_{tg}, D_{task} in the Decoder. Fig. 2 shows an example for workflow G_2 and a scheduled task v_3^2 at the current time step. For the task graph, each node represents a workflow task, including task workload, deadline, and the index of the assigned servers. The directed edges between nodes represent the partial order relationship between tasks, and the edge weight is the amount of data transfer between tasks. The task graph at time step t is built with the unscheduled task v_i^w.

Similarly, the resource is organized into an undirected graph according to their spatial relationships by the graph embedding model $model_r$. The main difference between the two models lies in the different numbers of layers of GCNs. To facilitate representation, we use constructed server nodes $\mathcal{V}_{j,0}^k$ to represent cluster nodes $\mathcal{H}_j^k$, and $\mathcal{V}_{0,0}^k$ to represent data center nodes $\mathcal{D}_k$. Each resource graph $G_t^{server} = (V_t^{server}, E_t^{server})$ at time step t includes all servers, clusters, and data center nodes, but the features of each node vary. Therefore, the number of nodes in each resource graph G_t^{server}, denoted as $|V_t^{server}| = \overline{\omega} + \sum_{k=1}^{M} \mu_k + M$, remains fixed. If a node is a server, its features include processing frequencies, power, and busy time; if it is a cluster, the feature values are the average features of all server on the cluster; if it is a DC, the feature is the average features of all cluster in the DC. For $model_r$ the output dimensions of each layer in the Encoder are all D_{rg}, and the output dimensions of the first two layers in the Decoder are D_{rg}, and the output dimension of the last layer is D_{server} . Fig. 3 shows the resource diagram at time step t = 2 in the above example with two data centers, each with two clusters, and each cluster with two servers.

In order to eliminate the scale difference of input data, the maximum and minimum value normalization is used to process the node features and edge weights in the task graph and resource graph to ensure the corresponding values within the range of [0,1]. For time step t and its unscheduled task v_i^w, given the embedding vector $\vec{X}_{task;w,i'} = (X_{task;w,i',[1]}, \ldots, X_{task;w,i',[D_{tn}]})$ of the task node $v_{i'}^w \in E_{w,i}^{task}$, the task graph embedding vector $\vec{S}_{task}$ is represented by the mean of the embedding vectors of all task nodes:

$$\vec{S}_{task} = \begin{bmatrix} \frac{1}{|V_{w,i}^{task}|} \sum_{v_{i'}^w \in V_{w,i}^{task}} X_{task;w,i',[1]} & \cdots \\ \frac{1}{|V_{w,i}^{task}|} \sum_{v_{i'}^w \in V_{w,i}^{task}} X_{task;w,i',[D_{tn}]} & \cdots \end{bmatrix} \tag{27}$$

At time t, given the embedding vector $\vec{X}_{server;k,j,l} = (X_{server;k,j,l,[1]}, \ldots, X_{server;k,j,l,[D_{rn}]})$ of the resource node $v_{j,l}^k \in E_t^{server}$, the resource graph embedding vector $\vec{S}_{server}$ is obtained by concatenating embedding vectors of all DC and cluster nodes: unnumbered equation shown at the bottom of the next page.

The two graph embedding models are trained by the graph autoencoder [27], [28].

2) Actor-Critic Framework for Workflows and Resources: To provide optimal resource allocation actions A_t at time t based on its real-time state in the MDP $\mathcal{M}$, we use the Actor-Critic framework [29], which combines two reinforcement learning methods: *value-based* and *policy gradient-based*. The policy network serves as the Actor, while a value function network, Critic, evaluates the quality of actions selected by the Actor. Both the Actor and Critic networks are implemented by a simple Multilayer Perceptron [30].

The network structure of the Actor-Critic framework is described in Fig. 4. The two networks share a state encoding module, which consists of two feedforward layers. This module transforms the input state vector into a hidden representation to both networks. The output dimensions of the two layers are $D_{encoder}(1)$ and $D_{encoder}(2)$, respectively. Subsequently, the Actor and Critic networks individually pass through two more feedforward layers to output c-dimension action vectors and 1-dimension action value estimation, respectively.

To select the most suitable DC $\mathcal{D}_k$ from M DCs and choose the optimal server from the ω^k servers in the DC, the Actor network outputs an action vector representing a probability distribution over the available actions. It can be further divided into sub-action probability vectors for selecting the DC and server. Sampling from these two vectors, it yields the indexes of selected DC and server. Since the output dimension of a neural network determines the number of parameters and connections, it must be fixed. When selecting the server, we consider the maximum number of servers ω_{max} among all DCs to compute the upper limit of the server index range. We use the Action Masking technique [31] to mask out non-effective actions beyond the range of the selected DC's server indexes to ensure that the

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

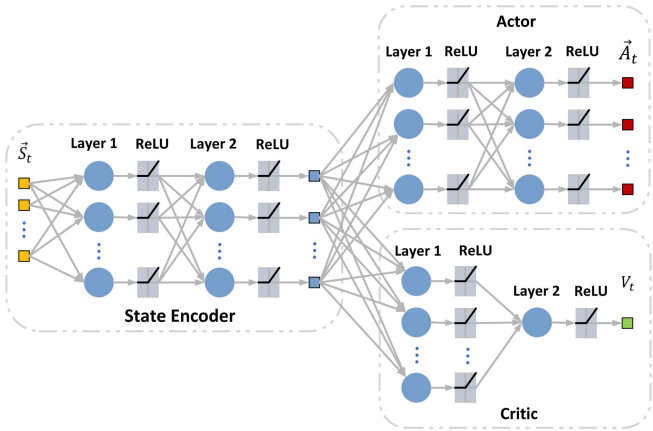


Fig. 4. Actor-Critic network architecture.

dimension D_{action} of the action vector can be fixed, calculated as: $D_{action} = M + \omega_{max}$.

The action masking technique is employed to improve the efficiency and stability of the policy by limiting the action space. For a given sub-action of selecting DC $\mathcal{D}_k$, the sub-action masking for selecting the server is determined by the maximum number of servers in the selected DC. A masking vector $\vec{Y}_{mask}$ of dimension ω_{max} is constructed as follows, where all elements with indexes exceeding ω^k are set to 0:

$$\vec{Y}_{mask} = \left[\overbrace{1 \quad \cdots \quad 1}^{\omega^k} \quad \overbrace{0 \quad \cdots \quad 0}^{\omega_{max}-\omega^k}\right] \tag{28}$$

The input dimension D_{state} of the Actor and Critic networks is fixed and equals to the sum of dimensions of the task graph embedding vector, resource graph embedding vector, electricity price vector, and algorithm parameter vector: $D_{state} = D_{task} + D_{server} + D_{price} + D_{params}$. Specifically, at the output layer of the policy network, the original action probability vector is multiplied element-wise with a masking vector containing 0 or 1, which restricts the number of available actions for different states. This modification only affects the value of the policy gradient and does not impact the internal structure or parameters of the policy network. Therefore, the PPO framework remains applicable.

The RAPPO algorithm is proposed to optimize the network parameters of both the Actor and Critic networks in Algorithm 4. It collects a batch of trajectories by the previous policy during each iteration. Then, it calculates the advantage function values $\hat{A}_t$ by the Generalized Advantage Estimation (GAE) method [32] and trains the Actor and Critic networks by gradient descent-based optimization. The advantage function describes the difference between the action value function and the state value function, reflecting the relative advantage or disadvantage of taking a certain action in a particular state compared to the average level. For resource allocation, the GAE advantage estimation $\hat{A}_t$ is defined as follows:

Algorithm 4: Resource Allocation PPO (RAPPO).

Input: Environment env; Actor π_θ; Critic V_ϕ; PPO clipping coefficient ϵ; GAE hyperparameter λ; discount factor γ; training iterations $iters$; trajectory length $steps$; number of epochs $epochs$; Critic loss weight vf_coef; entropy loss weight ent_coef; batch size for parameter updates $batches$; optimizer $optim$
. **Output:** Policy network π_θ
for $it = 1$ *to* $iters$ **do**
  Interact with the environment for $steps$ time steps, collect a batch of trajectories $trajs$;
  for $t = 1$ *to* $steps$ **do**
    $(S_t, R_t, S_{t+1}) \leftarrow trajs(t)$;
    Calculate GAE $\hat{A}_t$ by Eq. (29);
  for $i = 1$ *to* $epochs$ **do**
    $t \leftarrow 1$;
    while $t \leq steps$ **do**
      for $j = 1$ *to* $batches$ **do**
        Calculate the policy ratio $r_t(\theta)$ by Eq. (32);
        Calculate the loss $\mathcal{L}_\pi(\theta, t)$ by Eq. (31);
        $t \leftarrow t + 1$;
      Calculate the policy loss term $\mathcal{L}_\pi(\theta)$ by Eq.(30);
      Calculate the loss term $\mathcal{L}_V(\phi)$ by Eq. (33);
      Calculate the loss term $\mathcal{L}_{ent}(\theta)$ by Eq.(34);
      Calculate the loss by Eq.(35);
      $optim$.Step$(\theta, \phi, \Delta_{\theta,\phi}\mathcal{L}(\theta, \phi))$;
      $t \leftarrow t + batches$;

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{29}$$

where $\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t)$ is the temporal difference (TD) error at time step t, R_t is the reward, γ is the discount factor, and $V(S_t), V(S_{t+1})$ are the value function estimates for states S_t and S_{t+1}, respectively. λ is the GAE hyperparameter to control the trade-off between bias and variance, with a value between 0 and 1. When λ is close to 0, GAE produces lower variance but higher bias. Conversely, when λ is close to 1, it produces higher variance but lower bias. The term $(\gamma\lambda)^l$ is a discount factor that reduces the weight of future TD errors in the weighted accumulation process. GAE combines the weighted sum of TD errors over multiple time steps to estimate the advantages.

The RAPPO algorithm balances bias and variance during the estimation process, enhancing the stability and convergence speed of the policy gradient algorithm. RAPPO optimizes the Actor network by maximizing the clipped surrogate loss function, as defined in (30):

$$\mathcal{L}_\pi(\theta) = \frac{1}{batches} \sum_{t=t_0}^{t_0+batches} \mathcal{L}_\pi(\theta, t) \tag{30}$$

$$\left[\overbrace{\vec{X}_{server;1,1,0} \cdots \vec{X}_{server;M,\mu_M,0}}^{\sum_{k=1}^{M}\mu_k} \quad \overbrace{\vec{X}_{server;1,0,0} \cdots \vec{X}_{server;M,0,0}}^{M}\right].$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

$$\mathcal{L}_\pi(\theta, t) = \hat{E}_t\left[\hat{A}_t \cdot \min\{r_t(\theta), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\}\right] \tag{31}$$

where θ is the parameter of the policy network, $batches$ is the number of samples in a training batch, t_0 is the starting time step of the batch samples, and $r_t(\theta)$ is the probability ratio between the new and old policies, defined by

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{32}$$

where $\pi_\theta(a_t|s_t)$ is the probability of action a_t given state s_t when the policy parameter is θ. $\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the corresponding probability under the previous policy. ϵ is a hyperparameter that controls the size of policy updates and is typically set to 0.1 or 0.2. The loss function helps prevent overly large policy updates, thereby maintaining algorithm stability. By using $\min\{r_t(\theta), \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\}$ in the loss function, RAPPO constrains the range of the probability ratio $r_t(\theta)$ to fluctuate within the interval $[1-\epsilon, 1+\epsilon]$. To ensure training efficiency and stability while reducing training costs, the policy network is updated by a subset of the collected trajectories.

The goal of the Critic network is to accurately estimate the state value function $V(S_t)$. Mean squared error is used to measure the difference between the value function estimates and actual values:

$$\mathcal{L}_V(\phi) = \frac{1}{batches}\sum_{t=t_0}^{t_0+batches}(V_\phi(S_t) - G_t)^2 \tag{33}$$

where ϕ are the parameters of the Critic network, $V_\phi(S_t)$ denotes the value function estimate by the Critic network for state S_t, and G_t is the accumulated reward according to (26). The Critic loss aims to minimize the squared difference between the predicted values and the observed rewards, improving the accuracy of the value function estimation. To prevent the policy network from prematurely exploiting the explored state space and getting stuck in local optima, RAPPO uses an entropy loss function, $\mathcal{L}_{\text{ent}}(\theta)$, to balance the trade-off between exploration and exploitation as:

$$\mathcal{L}_{\text{ent}}(\theta) = -\mathbb{E}_{S_t\sim\rho(\cdot), A_t\sim\pi_\theta(\cdot|S_t)}\left[\log \pi_\theta(A_t|S_t)\right] \tag{34}$$

Although the exact state distribution $\rho(\cdot)$ is unknown, Monte Carlo sampling is employed by generating approximately instances based on the policy $\pi_\theta(A_t|S_t)$. By maximizing $\mathcal{L}_{\text{ent}}$, the policy network is encouraged to have a more uniform distribution of probabilities over different actions, leading to a more comprehensive exploration of the environment and facilitating the search for a global optimal policy. Since the Actor and Critic networks share a common State Encoder during training, the gradients are backpropagated by the same loss function. Therefore, the total loss function $\mathcal{L}(\theta, \phi)$ is defined as a weighted sum of the three loss terms:

$$\mathcal{L}(\theta, \phi) = -\mathcal{L}_\pi(\theta) + \text{vf_coef}\mathcal{L}_V(\phi) - \text{ent_coef}\mathcal{L}_{\text{ent}}(\theta) \tag{35}$$

where vf_coef and ent_coef are weight coefficients.

Algorithm 5: DARA.

Input: Task v_i^w
Output: Allocated DC $\mathcal{D}_k$ and server $\mathcal{V}_{j*,l*}^k$
Calculate $\hat{T}_{w,i}^{EST}$ of the task by Eq. (12);
$k \leftarrow argmin_{k=1,\ldots,M}\left\{p_k(\hat{T}_{w,i}^{EST})\right\}$;
$(j^*, l^*) \leftarrow argmax_{j=1,\ldots,\mu_k\ l=1,\ldots,\omega_j^k}\{\frac{\xi^k{}_{j,l}}{P_{k,j,l}}\}$;
Calculate the electricity cost $C_{w,i}^*$ by Eq. (5);
for $j = 1$ *to* μ_k **do**
  for $l = 1$ *to* ω_j^k **do**
    Calculate $T_{w,i;k,j,l}^{EFT}$ of task v_i^w on $\mathcal{V}_{j,l}^k$ by Eq. (36);
    if $T_{w,i;k,j,l}^{EFT} \le d_{w,i}$ **then**
      Calculate $C_{w,i}$ on $\mathcal{V}_{j,l}^k$ by Eq. (5);
      if $C_{w,i} \le C_{w,i}^*$ **then**
        $(j^*, l^*) \leftarrow (j, l)$;
        $C_{w,i}^* \leftarrow C_{w,i}$;

F. Reserve Strategy

When the confidence level of the action provided by the policy network is below the threshold, it is necessary to execute a reserve strategy. A heuristic resource allocation algorithm, Deadline Assured Resource Allocation (DARA) in Algorithm 5 is proposed as the reserve strategy adopted in the CCRA algorithm. DARA first selects a DC $\mathcal{D}_k$ with the lowest electricity price at the earliest possible start time $\hat{T}_{w,i}^{EST}$ of the task. Then, it searches for a server within that DC that can meet the task's deadline constraint while minimizing the electricity cost. If no server satisfies the requirements, DARA selects the server with the highest performance-to-power ratio. The worst-case time complexity for Algorithm 5 is $O(\max_{k=1,\ldots,M}\{\omega^k\})$. Since ECMWS schedules tasks based on their topological order in the workflow, DARA ensures that all direct predecessor tasks of task v_i^w are completed when allocating resources with the completion time $T_{w,i}^F$. Based on $T_{w,i}^F$ and $T_{k,j,l}^{avail}$ of server $\mathcal{V}_{j,l}^k$, DARA calculates the earliest start time $T_{w,i;k,j,l}^{EST}$ and earliest finish time $T_{w,i;k,j,l}^{EFT}$ of task v_i^w:

$$T_{w,i;k,j,l}^{EFT} = T_{w,i;k,j,l}^{EST} + T_{w,i}^D \tag{36}$$

where $T_{w,i;k,j,l}^{EST} = \max\{T_{k,j,l}^{avail}, \max_{v_{i'}^w \in PR_i^w} T_{w,i'}^F\}$ and $T_{k,j,l}^{avail} = \max_{w'=1,\ldots,N; i'=1,\ldots,n_{w'}; v_{i'}^{w'} \neq v_i^w} x_{w',i';k,j,l} \times T_{w',i'}^F$. $T_{k,j,l}^{avail}$ is defined as the latest completion time among all tasks allocated to $\mathcal{V}_{j,l}^k$. For the same example, the confidence level of the Actor-Critic framework is bigger than the threshold for workflow 2. The resource is allocated as $v_6^2 : V_{2,1}^1$, $v_5^2 : V_{1,2}^1$, $v_2^2 : V_{2,2}^1$, $v_4^2 : V_{1,1}^1$, $v_3^2 : V_{1,1}^1$, and $v_1^2 : V_{2,1}^1$.

V. Experimental Analysis

The cross-domain system consists of four DCs located in California (United States), Toronto Ontario (Canada), London (United Kingdom), and Munich (Germany). Based on the real data in [33], Fig. 5 shows the fluctuation of electricity prices in different DCs. The dashed lines depict the magnitude of the fluctuations in the electricity price of each data center. D_1 has the largest fluctuation and the peak and off-peak periods of electricity prices vary for each DC.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

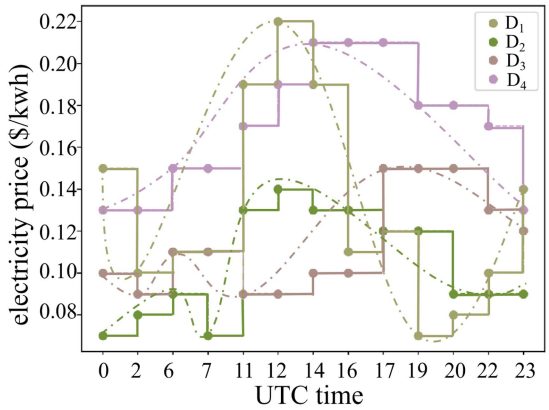


Fig. 5. Electricity price fluctuations in the four DCs.

TABLE II
DC CLUSTER CONFIGURATIONS

Cluster Type	Cluster1	Cluster2	Cluster3	Cluster4	Cluster5
GHz/Core	2.5	2.5	3	3	4
Cores#	4	8	4	8	4
MIPS/Core	500	500	600	600	800
Total MIPS	2,000	4,000	2,400	4,800	3,200
Power/Core	50W	50W	100W	100W	200W
Total Power	200W	400W	400W	800W	800W

TABLE III
SERVER CONFIGURATIONS

Server Type	Server1	Server2	Server3	Server4	Server5	Server6
Cores#	1	1	1	1	1	1
MIPS/Core	500	250	600	300	800	400
Power/Core	50W	25W	100W	50W	200W	100W
Deployment Cluster	1/2	1/2	3/4	3/4	5	5

To test the adaptability of the ECMWS algorithm to the volatility of electricity prices at a larger time scale while avoiding excessive experiment costs, the termination time T^{term} of the cross-domain DC system is set to one day, i.e., $3,600 \times 24 = 86,400$ seconds, and the start time is set to 0:00 UCT time. The time slot length τ is set to 10 minutes, i.e., 600 seconds. Each DC consists of five types of clusters including CPU cores, processing speed, and power which is shown in Table II. The number of different types of servers in each DC is randomly generated. Six types of servers with specified deployment cluster types are set, consistent with the assumptions of relevant studies [34], [35], facilitating the determination of the power consumption of each server. The specific configurations are shown in Table III.

To evaluate the performance of Algorithm 1, Relative Percentage Deviation (RPD) is employed to compare the electricity costs generated by different algorithm parameter and component combinations, calculated by:

$$RPD(\%) = \frac{Z - Z^*}{Z^*} \times 100\% \tag{37}$$

where Z represents the objective function value, i.e., the electricity cost calculated by (4). Z^* represents the minimum electricity cost generated by all algorithms when executing the corresponding workflow instances. We employ ANOVA (Analysis of Variance) to analyze the impact of different embedding vector dimension values on the inference performance of the graph embedding models.

A. Parameter Calibration

For the considered problem in this paper, we follow the approach in [7] to randomly generate workflow applications with various sizes $N \in \{40, 60, 80, 100, 120\}$ and numbers of tasks $n \in \{50, 100, 200, 500\}$. For each size and number of tasks, we generate 10 workflow instances and randomly assign arrival times $T_w^{submit} \in [0, T^{term})$. The deadlines d_w for the workflow applications are determined by $d_w = \hat{T}_w^{EFT} \times (1 + \rho)$. $\rho \in \{0.2, 0.4, 0.6, 0.8, 1\}$ is a parameter which controls the tightness of the workflow application deadlines. Consequently, a total of $5 \times 4 \times 5 \times 10 = 1,000$ random workflow application instances are generated for parameter calibration. For Policy Network Training, we construct a reinforcement learning environment based on the cross-domain DC configuration by utilizing the OpenAI Gym framework and the extend Cloudsim Plus [36] simulation platform. After experimental validations, the output dimension of the first encoder layer is 512, and the second encoder is 256. The first actor layer is 128 and the second actor layer is 227. The first critic layer is 128 and second critic layer is 1. The training of the policy network parameters is performed by the RAPPO algorithm with 100 epochs, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.1$.

Based on WfCommons [37], 2,000 workflow instances containing 500 tasks are randomly generated for the Epigenomics, Genome, and Montage types of scientific workflows [38]. Subsequently, the dataset is randomly divided into training, validation, and testing sets in a ratio of 7:2:1. To select a suitable dimension for the task graph embedding vector D_{tn}, four values $D_{tn} \in \{15, 25, 30, 35\}$, are validated, and the performance comparison results of the corresponding models are shown in Table IV. We use RPD to estimate the performance of different parameters, and a lower RPD represents better allocation with lower consumption of electricity cost. We can see that $D_{tn} = 25$ outperforms the other values, indicating that it has more stable performance on test samples. To determine the output dimensions of the task graph embedding model, four different values, i.e., $D_{tg} \in \{35, 50, 65, 70\}$, are compared in Table IV. $D_{tg} = 50$ outperforms better than others because of its optimal balance between complexity and expressive power. A dataset of 2,000 resource graphs is generated based on the cross-domain DC configurations by randomly setting the busy time of servers. The epoch is set as 1,000, and the batch size is set as 10. Five different dimensions for the resource graph embedding vectors $D_{server} \in \{4, 8, 12, 16, 20\}$ are explored, and their corresponding model performance is compared in Table IV. We can observe that $D_{rn} = 8$ exhibits significant advantages over higher-dimensional values. It demonstrates more stable performance which benefits from the proper expressive computing power. Therefore, the dimension of the resource graph embedding vector is set to 8.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
PARAMETER CALIBARATION WITH A 95% TUKEY HSD CONFIDENCE INTERVAL

RPD(%)	**Task graph embedding vector(D_{tn})**				**Encoder input layer(D_{tg})**				**Resource graph embedding(D_{server})**				
	15	25	30	35	35	50	65	70	4	8	12	16	20
Lowerbound	-499	-421	3003	1891	222	-550	1783	2699	-130	-52	1021	1087	712
Uppebound	2,098	1,723	5,031	4,098	2,920	1,498	3,894	4,867	174	169	1263	1332	969
RPD (%)	**CWS factor ($\alpha_1, \alpha_2, \alpha_3$)**			**BLDP bottleneck(β)**				**Task sorting (TS)**			**Task sorting ($Conf_{thresh}$)**		
	(O.4,0.2,0.4)	(O.4,0.3,0.3)	(O.5,0.2,0.3)	1.5	2	4	8	TS_1	TS_2	TS_3	0.2	0.5	0.8
Lowerbound	9.6	11.0	11.9	13.3	12.0	10.1	14.5	10.3	10.4	9.4	19.0	14.6	23.0
Upperbound	10.6	12.0	12.9	14.4	13.1	11.2	15.6	11.3	11.4	10.4	19.9	15.5	23.9

TABLE V
ABLATION STUDY

RPD(%)	ECMWS	-WS	-TS	-RAPPO
Lowerbound	10.7	12.6	13.2	15.2
Uppebound	11.8	13.7	14.3	16.3

For the workflow ordering algorithm, we test 15 valid combinations of $\alpha_1, \alpha_2, \alpha_3 \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$ based on the weight constraint relationship in (17). The parameter β for BLDP is selected from $\{1.5, 2, 4, 8\}$. Three task sorting strategies, TS_1, TS_2, and TS_3, are considered. $Conf_{thresh}$, which determines the confidence threshold for action selection, is motivated to balance the trade-off between exploration and exploitation. Given its range spans from 0 to 1, we systematically test $Conf_{thresh}$ at 0.2, 0.5, and 0.8. For each workflow, $15 \times 4 \times 3 \times 3 = 540$ experiments need to be conducted. The parameter calibration requires a total of $1,000 \times 540 = 540,000$ experiments to comprehensively compare different components and parameter combinations for the optimal algorithm configuration. The RPD values of the ECMWS algorithm are shown in Table IV within a 95% Tukey HSD confidence interval. The performance of the CWS sorting in Algorithm 2 under different combinations of factor weights $(\alpha_1, \alpha_2, \alpha_3)$ is presented in Table IV. Among these combinations, $(\alpha_1, \alpha_2, \alpha_3) = (0.4, 0.2, 0.4)$ exhibits a significant superiority over others. The performance of different task sorting strategies is depicted in Table IV. Among these strategies, TS_3 demonstrates a significant advantage over the others because the perception of bottleneck layers with higher ranks ensures that tasks with larger ranks are executed before tasks with smaller ranks. The table also presents the performance of the CCRA algorithm under different confidence thresholds. Smaller confidence thresholds (e.g., 0.2) tend to prioritize the output actions of the policy network, while larger confidence thresholds (e.g., 0.8) have a higher probability of executing backup policies. It can be observed that RPD is the lowest when $Conf_{thresh} = 0.5$ due to the balance between the policy network and the backup policy.

B. Ablation Study

To better verify our contributions, we conduct ablation studies on the ECMWS method and the workload setting is the same as Section V-A. We primarily investigate the impacts of the Workflow Sequencing module, the Task Sequencing module and the RAPPO module. The findings, as illustrated in Table V, collectively demonstrate the importance of these modules in minimizing scheduling costs by ECMWS. We begin with the complete ECMWS method and then remove the Workflow Sequencing module (-WS) with a random sorting strategy, which result in an approximate 18% increase in RPD, indicating that our tailored workflow sorting based on slack time, workload, and resource contention factors significantly reduces scheduling expenses. Next, we maintain all other modules and randomize the task sequencing process (-TS), leading to a 22% rise in RPD. This suggests that the absence of sub-deadline partitioning in task ordering can lead to resource misallocation and potential deadline violations. Finally, replacing RAPPO (-RAPPO) with a load balancing method, which disregards regional electricity cost variances, causes a substantial 41% increase in RPD. This underscores the cost inefficiency when ignoring different prices.

C. Algorithm Comparison

To examine the performance of ECMWS, we use the Epigenomics [39] and Genome [40] scientific workflows. These workflows are suitable for highly pipelined applications with multiple pipelines operating on independent chunks of data in parallel. Both have complex structures of workflows and are widely used in the existing studies [7], [38]. To the best of our knowledge, there is no existing algorithm for the problem under study. We compare the proposed algorithm to HEFT [25], ECWSD [7], and DEWS [41] which are modified for better contrast. ECWSD is a static scheduling algorithm designed for optimizing the electricity cost of multiple workflows, while HEFT and DEWS only consider a single workflow. We have made certain modifications to the three algorithms, which involve scheduling a batch of workflows that arrive within each time slot at the end of the time slot.

Fig. 6(a) presents the mean RPD of electricity cost for Epigenomics workflows with different numbers of workflows by comparing these four algorithms. In Fig. 6(a), the ECMWS algorithm consistently outperforms the other three algorithms as the number of workflows increases. From $N = 40$ to 80, the advantage of ECMWS over the others expands while from $N = 80$ to 120, it starts to diminish. The performance improvement ratio of the ECMWS algorithm decreases when the number of workflow instances is high. This is because that with the increasing number of workflow instances, the system undergoes more diverse state changes, which may lead to unexplored states in the strategy network. It can further lead to being trapped in local optima. For real-world large-scale workflow deployments, to explore the performance for the proposed algorithm, we can divide a large-scale workflow into sub-workflows with the size less than 80. Fig. 6(b) illustrates the mean RPD of electricity cost for four algorithms on Epigenomics workflows with different task number configurations. DEWS, HEFT, and ECWSD are initially close but

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

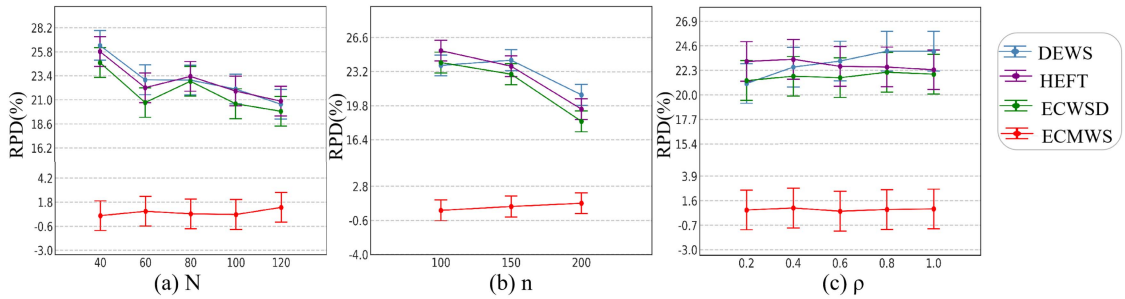


Fig. 6. Algorithm comparison for parameters N, n, ρ on Epigenomics with 95.0% Tukey HSD confidence level intervals.

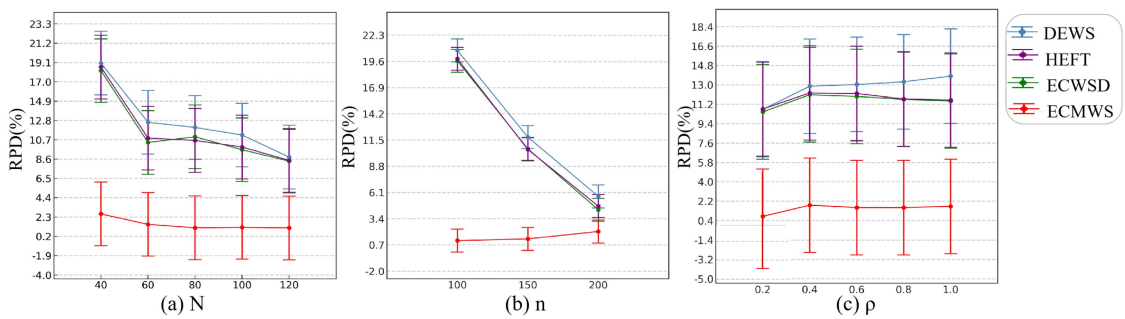


Fig. 7. Algorithm comparison for parameters N, n, on Genome with 95.0% Tukey HSD confidence level intervals.

diverge as n increases. In contrast, ECMWS demonstrates minor fluctuations and steady improvement in performance because the allocation ability of the ECMWS algorithm is improved for complex system states by graph embedding technique. The mean RPD of electricity cost for four algorithms of Epigenomics workflows at different levels of workflow deadline constraints, is shown in Fig. 6(c). As ρ increases, the workflow deadline constraints are relaxed. The overall performance of all algorithms improves because workflow tasks must be executed on servers with higher processing frequencies to meet tighter workflow deadline constraints. Regardless of the degree of relaxation in the deadline constraints, ECMWS consistently outperforms the HEFT, ECWSD and DEWS algorithms. Overall, ECMWS is more reliable across different scenarios due to its consistent low-RPD performance.

For Genome workflows, Fig. 7(a) presents the mean RPD of electricity cost with different numbers of workflows. As N increases from 40 to 120, the RPD of all four methods declines. DEWS, HEFT, and ECMWS exhibit similar trends, with initial RPD values of approximately 19.1%, 18.5% and 18.0%, respectively, when $N = 40$. It decreases significantly when $N = 60$. Finally, it gradually declines and approaches each other. ECWSD maintains a consistently low and stable RPD throughout the change of N, reflecting its stability for different number of workflows. In Fig. 7(b), as the number of tasks increases from 100 to 200, the RPD of HEFT, ECWSD and DEWS overlaps to a large extent in the early stage, while ECWSD drops more rapidly in the later stage and has the lowest RPD when n=200. In contrast, ECMWS exhibits minor fluctuations and consistent performance improvement as n increases due to the combination of policy networks and backup policies, which accurately predict and adjust task priorities for resource allocation. In Fig. 7(c), as ρ increases from 0.2 to 1.0, the RPD of the four methods changes relatively smoothly. The RPD of HEFT, ECWSD and DEWS are close to each other. Overall, ECMWS demonstrates remarkable advantages, maintaining the lowest and most stable RPD under different parameter conditions, indicating its reliable performance with small relative deviations. In contrast, HEFT, ECWSD and DEWS show larger fluctuations in RPD values under different parameters, especially HEFT, suggesting poor stability due to its simple greedy strategy.

TABLE VI
COMPARISON ON ELECTRICITY COST AND CPU TIME

Algorithms	Epigenomic		Genome	
	Cost	CPU time(s)	Cost	CPU time(s)
ECMWS	0.766%	18637.231	1.516%	19820.938
ECWSD	21.755%	3644.691	11.531%	4212.733
DEWS	23.007%	9416.966	12.748%	9917.856
HEFT	22.824%	188.895	11.700%	200.195

To compare the efficiency among these algorithms, Table VI provides an overview of the electricity cost and CPU time of the comparing algorithms. According to the table, the ECMWS algorithm achieves the best performance in terms of electricity cost, albeit with a longer execution time. However, the execution time of the ECMWS algorithm falls within an acceptable range for practical implementation (seconds).

VI. CONCLUSION

In this paper, we address the cross-domain multi-workflow allocation problem in GDCs with fixed server frequency and

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

power to minimize the electricity cost. A cost-aware multi-workflow scheduling algorithm named ECMWS is proposed, which consists of the Contention-aware Workflow Sequencing, Bottleneck-Layer-aware sub-Deadline Partitioning, Task Sequencing and Resource Allocation. The experimental results demonstrate that the proposed ECMWS algorithm consistently outperforms the baseline algorithms while maintaining acceptable execution efficiency. For future works, we will focus on exploring the cross-domain multi-workflow scheduling problem with multiple primary nodes. For more than one masters, it needs more interactions among masters that increases the latency. The system's complexity is increased to manage and monitor multiple primary nodes and their interactions. In addition, the topology structure of these masters would affect the algorithm complexity.

References

[1] M. Ghamkhari, A. Wierman, and H. Mohsenian-Rad, "Energy portfolio optimization of data centers," *IEEE Trans. Smart Grid*, vol. 8, no. 4, pp. 1898–1910, Jul. 2017.

[2] H. Wang, J. Huang, X. Lin, and H. Mohsenian-Rad, "Proactive demand response for data centers: A win-win solution," *IEEE Trans. Smart Grid*, vol. 7, no. 3, pp. 1584–1596, May 2016.

[3] J. Doyle, R. Shorten, and D. O'Mahony, "Stratus: Load balancing the cloud for carbon emissions control," *IEEE Trans. Cloud Comput.*, vol. 1, no. 1, pp. 116–128, Jan./Jun. 2013.

[4] J. C. Jacob et al., "Montage: A grid portal and software toolkit for science-grade astronomical image mosaicking," *Int. J. Comput. Sci. Eng.*, vol. 4, no. 2, pp. 73–87, Jul. 2009.

[5] International Energy Agency, "World Energy Outlook," 2022. [Online]. Availaible: https://iea.blob.core.windows.net/assets/830fe099-5530-48f2-a7c1-11f35d510983/WorldEnergyOutlook2022.pdf

[6] A. Jayanetti, S. Halgamuge, and R. Buyya, "Multi-agent deep reinforcement learning framework for renewable energy-aware workflow scheduling on distributed cloud data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 604–615, Apr. 2024.

[7] X. Li, W. Yu, R. Ruiz, and J. Zhu, "Energy-aware cloud workflow applications scheduling with geo-distributed data," *IEEE Trans. Serv. Comput.*, vol. 15, no. 2, pp. 891–903, Mar./Apr. 2022.

[8] Y. Xie, F.-X. Gui, W.-J. Wang, and C.-F. Chien, "A two-stage multi-population genetic algorithm with heuristics for workflow scheduling in heterogeneous distributed computing environments," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1446–1460, Second Quarter 2023.

[9] C. Powell, K. Miura, and M. Munetomo, "Many-constraint and many-objective optimization with bias index for intercloud multi-workflow resource provisioning," in *Proc. Joint 12th Intl. Conf. Soft Comput. Intell. Syst. 23rd Intl. Symp. Adv. Intell. Syst.*, 2022, pp. 1–8.

[10] W. Ahmad, B. Alam, and A. Atman, "An energy-efficient Big Data workflow scheduling algorithm under budget constraints for heterogeneous cloud environment," *J. Supercomputing*, vol. 77, no. 10, pp. 11946–11985, Oct. 2021.

[11] H. Zhang, Y. Zhang, S. Wang, and J. Lu, "Electricity cost minimization for workflows scheduling in geo-distributed data centers," in *Proc. 17th Int. Conf. Serv. Sci.*, 2024, pp. 30–45.

[12] N. Rizvi and D. Ramesh, "Fair budget constrained workflow scheduling approach for heterogeneous clouds," *Cluster Comput.*, vol. 23, no. 4, pp. 3185–3201, Dec. 2020.

[13] I. Attiya, M. A. Elaziz, L. Abualigah, T. N. Nguyen, and A. A. A. El-Latif, "An improved hybrid swarm intelligence for scheduling IoT application tasks in the cloud," *IEEE Trans. Ind. Informat.*, vol. 18, no. 9, pp. 6264–6272, Sep. 2022.

[14] S. Ijaz, E. U. Munir, S. G. Ahmad, M. M. Rafique, and O. F. Rana, "Energy-makespan optimization of workflow scheduling in fog–cloud computing," *Computing*, vol. 103, no. 9, pp. 2033–2059, Sep. 2021.

[15] B. Feng, Z. Ding, X. Zhou, and C. Jiang, "Heterogeneity-aware proactive elastic resource allocation for serverless applications," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2473–2487, Sep./Oct. 2024.

[16] M. A. Rodriguez and R. Buyya, "Scheduling dynamic workloads in multi-tenant scientific workflow as a service platforms," *Future Gener. Comput. Syst.*, vol. 79, no. P2, pp. 739–750, Feb. 2018.

[17] J. Zhang, L. Cheng, C. Liu, Z. Zhao, and Y. Mao, "Cost-aware scheduling systems for real-time workflows in cloud: An approach based on genetic algorithm and deep reinforcement learning," *Expert Syst. Appl.*, vol. 234, 2023, Art. no. 120972.

[18] J. Yan et al., "Energy-aware systems for real-time job scheduling in cloud data centers: A deep reinforcement learning approach," *Comput. Elect. Eng.*, vol. 99, 2022, Art. no. 107688.

[19] T. Dong, F. Xue, C. Xiao, and J. Zhang, "Deep reinforcement learning for dynamic workflow scheduling in cloud environment," in *Proc. IEEE Intl. Conf. Serv. Comput.*, 2021, pp. 107–115.

[20] N. Garg, D. Singh, and M. S. Goraya, "Energy and resource efficient workflow scheduling in a virtualized cloud environment," *Cluster Comput.*, vol. 24, pp. 767–797, 2020.

[21] B. Lin, W. Guo, N. Xiong, G. Chen, A. V. Vasilakos, and H. Zhang, "A pretreatment workflow scheduling approach for Big Data applications in multicloud environments," *IEEE Trans. Netw. Serv. Manage.*, vol. 13, no. 3, pp. 581–594, Sep. 2016.

[22] J. Zhang, Z. Yao, Y. Tu, and Y. Chen, "A survey of TCP congestion control algorithm," in *Proc. IEEE 5th Int. Conf. Signal Image Process.*, 2020, pp. 828–832.

[23] R. A. Howard, "Dynamic programming," *Manage. Sci.*, vol. 12, no. 5, pp. 317–348, 1966.

[24] G. Yao, Y. Ding, and K. Hao, "Using imbalance characteristic for fault-tolerant workflow scheduling in cloud systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3671–3683, Dec. 2017.

[25] H. Topcuouglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv: 1707.06347*.

[27] G. E. Hinton and R. R. Salakhutdinov, "Autoencoder," in *Encyclopedia of Machine Learning*. New York, NY, USA: Springer, 2014, pp. 1–9.

[28] D. Sun, D. Li, Z. Ding, X. Zhang, and J. Tang, "Dual-decoder graph autoencoder for unsupervised graph representation learning," *Knowl.-Based Syst.*, vol. 234, 2021, Art. no. 107564.

[29] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. 12th Int. Conf. Neural Inf. Process. Syst.*, 1999, pp. 1057–1063.

[30] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[31] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[32] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. 4th Int. Conf. Learn. Representations*, San Juan, Puerto Rico, 2016, pp. 1–14.

[33] A. Forestiero, C. Mastroianni, M. Meo, G. Papuzzo, and M. Sheikhalishahi, "Hierarchical approach for efficient workload management in geo-distributed data centers," *IEEE Trans. Green Commun. Netw.*, vol. 1, no. 1, pp. 97–111, Mar. 2017.

[34] H. Zhu, X. Li, L. Chen, and R. Ruiz, "Smart offloading computation-intensive delay-intensive tasks of real-time workflows in mobile edge computing," in *Proc. 2023 IEEE Int. Conf. Web Serv.*, Chicago, IL, USA, 2023, pp. 695–697.

[35] Z. Tong, J. Deng, J. Mei, Y. Zhang, and K. Li, "Multi-objective DAG task offloading in MEC environment based on federated DQN with automated hyperparameter optimization," *IEEE Trans. Serv. Comput.*, vol. 17, no. 6, pp. 3999–4012, Nov./Dec. 2024.

[36] M. C. Silva Filho, R. L. Oliveira, C. C. Monteiro, P. R. M. Inácio, and M. M. Freire, "CloudSim plus: A cloud computing simulation framework pursuing software engineering principles for improved modularity, extensibility and correctness," in *Proc. 2017 IFIP/IEEE Symp. Integr. Netw. Serv. Manage.*, 2017, pp. 400–406.

[37] T. A. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, no. C., pp. 16–27, Mar. 2022.

[38] G. Yao, X. Li, Q. Ren, and R. Ruiz, "Failure-aware elastic cloud workflow scheduling," *IEEE Trans. Serv. Comput.*, vol. 16, no. 3, pp. 1846–1859, May/Jun. 2023.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.

[39] S. Bharathi, A. Chervenak, E. Deelman, G. Mehta, M.-H. Su, and K. Vahi, "Characterization of scientific workflows," in *Proc. 3rd Workshop Workflows Support Large-Scale Sci.*, 2008, pp. 1–10.
[40] V. Prakash, S. Bawa, and L. Garg, "Multi-dependency and time based resource scheduling algorithm for scientific applications in cloud computing," *Electronics*, vol. 10, 2021, Art. no. 1320.
[41] M. Hussain, L.-F. Wei, A. Rehman, F. Abbas, A. Hussain, and M. Ali, "Deadline-constrained energy-aware workflow scheduling in geographically distributed cloud data centers," *Future Gener. Comput. Syst.*, vol. 132, pp. 211–222, 2022.

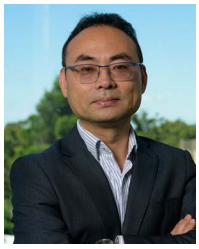
Shuang Wang received the BSc degree from the College of Sciences from Nanjing Agricultural University in 2015 and the PhD degree from the School of Computer Science and Engineering, Southeast University, Nanjing, China in 2020. She is currently a lecturer with the Southeast University. Her research has been published in international journals and conferences such as *IEEE Transactions on Computers, IEEE Trans. on Parallel and Distributed Systems, IEEE Trans. on Services Computing, IEEE Trans. on Network and Service Management* and ICSOC. Her main research interests include cloud computing, scheduling optimization, and Big Data.

He Zhang received the BSc degree from Chien-Shiung WU College, Southeast University in 2022. She is currently working toward the master's degree with the School of Computer Science and Engineering, Southeast University, Nanjing, China. Her main research interests focus on truth discovery and cloud computing.

Tianxing Wu received the PhD degree in software engineering from Southeast University, in 2018. He is an associate professor with the School of Computer Science and Engineering of Southeast University, China. He has published more than 60 papers in top-tier international conferences and journals, including SIGMOD, ICDE, AAAI, IJCAI, SIGIR, *IEEE Transactions on Knowledge and Data Engineering* and etc. His research interests include knowledge graph, knowledge representation and reasoning, and artificial intelligence applications.

Yueyou Zhang received the BSc degree from the School of Computer and Communication Engineering, the University of Science and Technology in Beijing in 2020and the master's degree from the School of Computer Science and Engineering, Southeast University in 2023. His main research interest focuses on cloud computing and resource scheduling.

Wei Emma Zhang received the PhD degree in computer science from the University of Adelaide. She is a senior lecturer and associate head of People and Culture with the School of Computer and Mathematical Sciences, The University of Adelaide. She is an ARC Industry Early Career Research fellow 2023-2026. She research interests include document summarization, adversarial attacks and Artificial Intelligence of Things. She has more than 100 publications and is among the top 100 authors worldwide, ranked by field-weighted citation impact, in the SciVal topic for Network Security.



Quan Z. Sheng received the PhD degree in computer science from the University of New South Wales (UNSW). He is a distinguished professor and head with the School of Computing, Macquarie University, Australia. His research interests include service computing, web technologies, machine learning and Internet of Things (IoT). He is the recipient of AMiner Most Influential Scholar Award in IoT in 2019, ARC (Australian Research Council) Future Fellowship in 2014, Chris Wallace Award for Outstanding Research Contribution in 2012, and Microsoft Research Fellowship in 2003. He is ranked by Microsoft Academic as one of the Most Impactful Authors in Services Computing (the 4th all time) in 2021. He is the vice chair of the Executive Committee of the *IEEE Technical Community on Services Computing* and a member of the Australian Computer Society Technical Advisory Board on IoT.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on December 14,2025 at 04:57:21 UTC from IEEE Xplore. Restrictions apply.